ReceivableOS — Security Self-Assessment

Competition	FinTechathon 2026, International Track
Topic	F — Agent Wallet (AI Agent Digital Wallet)
Repository	https://github.com/Khawpuneiei/ReceivableOS
Demo video	https://youtu.be/Bq-4jXATmx4
Live demo (real FISCO BCOS chain)	https://receivableos-chain.onrender.com
Live demo (mock ledger, backup)	https://receivableos-live.onrender.com
SME owner app	https://receivableos-sme.onrender.com
Assessed revision	`main` at commit `04627de`
Date	2026-10-20
Submission item	Security self-assessment report: permission-tier implementation and known-risk list

ReceivableOS turns an SME invoice into working capital. An OCR service reads the invoice, a policy service makes a credit decision, and an agent wallet draws the advance from a lender vault on FISCO BCOS. The wallet's design rule is: **LLM output never becomes a transaction directly.** Models and services propose typed decisions, deterministic code validates them, and the on-chain contracts have the final say. A human approves large payments.

---

1. Scope and method

1.1 What was assessed

Component	Path	Security role
Agent wallet service (FastAPI)	`services/agent-wallet/app/`	Off-chain guard layers L0 to L4, token checks, lifecycle, idempotency, audit log
Ledger adapters	`services/agent-wallet/ledger/mock.py`, `ledger/fisco.py`	Python mirror of the contracts (mock) and the FISCO BCOS client
Smart contracts (Solidity 0.8.11)	`services/agent-wallet/contracts/src/`	`AgentWallet`, `PolicyGuard`, `ReceivableRegistry`, `FinancingVault`, `MockStable` (tTHB). This is layer L5
OCR service	`services/ocr-ai/app/`	Document injection detection, fraud signals, keyed invoice fingerprint
Policy service	`services/policy-ai/app/`, `rules/policy.v1.yaml`	Hard rules before any model, credit decision, decision hash, mandate compiler
Orchestrator	`services/orchestrator/app/main.py`	Caller authentication, holder of the wallet's service tokens, mock lender market

Out of scope: the UI, the pitch material, and production infrastructure, which does not exist yet.

1.2 How it was assessed

1. **Code review against the docs.** Every control in this report was checked in the code at `04627de`. Where a document claims something the code does not do, the gap is listed in section 5.
2. **Unit and contract tests.** All four Python test suites were re-run for this report on 2026-10-19. The Foundry suite (`forge test`) could not be re-run in the assessment environment, so its count comes from the engineer's report and a count of the test functions in the source (section 6).

3. **Red-team evaluation.** `services/agent-wallet/eval/run_eval.py` runs 8 case sets: 216 attacks and 86 benign cases. Each case gets a fresh wallet. The unauthorized-payment check reads the ledger state after each case, not the guard's verdict. Each set was run in three configurations: scanner alone, chain alone (all off-chain layers bypassed), and the full stack. `eval/REDTEAM.md` was regenerated for this report in a scratch copy and matched the committed file line for line.
4. **On-chain evidence.** The contracts were deployed to FISCO BCOS 3.16.4 (a 4-node Air chain), and one full demo was run on them. `scripts/verify_fisco_hashes.py` confirmed every recorded hash against the chain. Revert cases were sent on chain through the `DEMO_MODE` bypass so the contracts had to refuse them alone.

---

2. Architecture of the controls (summary)

```
request -> L0 schema/currency/precision -> L1 decision provenance -> L2 injection scan + canary
        -> L3 mandate and spending checks -> L4 human-approval gate
        -> L5 on-chain: AgentWallet -> PolicyGuard -> FinancingVault / ReceivableRegistry
```

- L0 to L4 run in `app/guard/pipeline.py` (`check_draw`) and `app/wallet/lifecycle.py`. All checks run and every failure is reported. If any blocking check fails, nothing is sent to the ledger.
- L5 is in the contracts. The contracts use the same revert strings as the off-chain codes, so off-chain and on-chain refusals read the same.
- Funds sit in `FinancingVault`, not in the agent's account. The vault pays only the seller address bound to the receivable when it was registered. No request field, invoice text or model output can supply a payee address.

---

3. Permission-tier implementation

3.1 Roles and what each may do

"Off-chain" means the HTTP check in `services/agent-wallet/app/main.py`, done with `_check_token`. It fails closed when the token is not configured and compares tokens in constant time. "On-chain" means a Solidity modifier or `require`.

Role (holder in the demo)	May do	May not do	Where enforced	File / function
Owner (PM; the deployer key)	Set the mandate (`PolicyGuard.setMandate`), call and payee allowlists (`setCall`, `setPayee`), wire contracts once (`wire`), set the settlement payer (`FinancingVault.setSettlementPayer`), replace agent or approver (`AgentWallet.setAgent`, `setApprover`), pause, **unpause**, mint tTHB (`MockStable.mint`). HTTP: `/wallet/mandate/activate`, `/wallet/unpause`, `/wallet/pause` with `X-Owner-Token`	Confirm a payment (`confirm` is `onlyApprover`); call `execute`	Both	`PolicyGuard.onlyOwner`, `AgentWallet.onlyOwner`, `FinancingVault.onlyOwner`, `ReceivableRegistry.onlyOwner`; test `testOwnerCannotConfirmAPayment`; pytest `test_owner_cannot_release_and_the_queue_needs_the_approver_token`
Approver (Finance teammate)	Release or decline a held draw (`AgentWallet.confirm(id, approve)`), pause. HTTP: `/wallet/confirm/{tx_id}`, `/wallet/pause`, `GET /wallet/holds` with `X-Approver-Token`	Change the mandate, unpause, execute	Both	`AgentWallet.onlyApprover`, `pause()`; pytest `test_large_draw_needs_the_approver_token_not_the_owner`

Role (holder in the demo)	May do	May not do	Where enforced	File / function
Agent (wallet service key)	`AgentWallet.execute` and `anchorAudit` only. Through `execute`, only the allowlisted (target, selector) pairs: `Registry.register`, `Vault.draw`, `Vault.repay`. HTTP: `/wallet/register`, `/wallet/execute`, `/wallet/settle` with `X-Agent-Token`	Approve credit, confirm, pause or unpause, set the mandate, call any other contract or function	Both	`AgentWallet.onlyAgent`; `PolicyGuard.allowedCall` (`TargetNotAllowed`); tests `testOnlyTheAgentCanExecute`, `testTargetOutsideTheAllowlistReverts`; pytest `test_agent_endpoints_refuse_calls_without_the_agent_token`
Risk oracle (separate key; co-located in the MVP, see risk R2)	Credit approval (TX-2): `ReceivableRegistry.approve(fingerprint, limit, maxAprBps, decisionHash)`, once per receivable. HTTP: `/wallet/approve` with `X-Risk-Oracle-Token`, and the request must carry the decision it names	Register, draw, settle, change the mandate	Both	`ReceivableRegistry.onlyRiskOracle` (address immutable), `AlreadyApproved`; `WalletService._approve` (decision must be `approve`, hash must recompute, limit and fee must equal the decision's, PD at or below the mandate ceiling); tests `testOnlyTheRiskOracleApproves`, `testAnApprovalIsGivenOnce`; pytest `test_approve_without_the_risk_oracle_token_is_refused`, `test_approval_above_the_pd_ceiling_is_refused`
Lender (setup script)	Deposit liquidity (`FinancingVault.deposit`), claim its own earned fees (`claimFees`)	Draw or repay	On-chain	`FinancingVault.onlyWallet` on `draw` and `repay`; test `testOnlyTheWalletDraws`
Buyer / settlement payer (one demo account)	Holds the tTHB that settles the invoice and grants the vault an allowance	Nothing else. It does not sign individual settlements (risk R13)	On-chain	`FinancingVault.repay` pulls from `settlementPayer` via `transferFrom`
Seller (SME)	Receives the advance and the reserve at the address bound at registration	Hold any key in the system	Both	`ReceivableRegistry.register` requires `guard.payeeAllowed(seller)`; `Vault.draw` pays `r.seller` only
Contracts as callers	`AgentWallet` registers, draws and repays; `FinancingVault` and `AgentWallet` move receivable status; only `AgentWallet` records spend	—	On-chain	`onlyWallet` on `Registry.register`, `Vault.draw/repay`, `PolicyGuard.record/revalidate`; `setStatus` requires wallet or vault; tests `testOnlyTheWalletRegistersReceivables`, `testOnlyTheWalletOrVaultChangesStatus`, `testOnlyTheWalletRecordsSpend`, `testEachContractIsWiredOnce`
Orchestrator (PM's service)	Holds the agent and risk-oracle HTTP tokens and calls the wallet for the flow. Its own callers need `X-Orchestrator-Token`	—	Off-chain	`services/orchestrator/app/main.py` `check_caller` (fails closed when unset)
Unauthenticated	`GET /health`, `/wallet/mandate`, `/wallet/audit/*`, `/wallet/tx/{tx_id}` (read only)	Any write	Off-chain	`app/main.py`

3.2 Tiers by amount

All values are THB with 2 decimals and come from `config/mandate.demo.json`. The env values `WALLET_MAX_TX_AMOUNT` and `WALLET_DAILY_CAP` are platform ceilings that a mandate can only tighten (`Mandate.within_ceilings`).

Draw amount	Outcome	Where enforced
Below 837,500.00	Executes automatically if every L0 to L5 check passes	Off-chain L4 (`needs_approval`), on-chain `PolicyGuard.check` returns `false`
837,500.00 to 1,500,000.00 (threshold inclusive, `>=`)	`pending_approval`. The contract stores the exact call (target, calldata, amount, APR, fingerprint, decision hash), and only the approver can release that stored call. At confirm, `PolicyGuard.revalidate` re-runs target, expiry, per-tx cap and APR. The off-chain record expires after 24 hours (`ApprovalExpired`), and the draw is then declined on chain too	Both: `AgentWallet.execute` / `confirm`; `lifecycle.confirm`, `HOLD_TTL`
Above 1,500,000.00	Refused: `AmountAboveTxCap`	Both
Rolling 24 h total above 3,000,000.00 (held draws count)	Refused: `DailyCapExceeded`	Both. `PolicyGuard.record` charges held draws when they are proposed. On FISCO the service seeds its off-chain total from `PolicyGuard.rollingSpend()` at startup and refuses to start if the read fails
Any amount, when only the optional ML classifier flags the text and `INJECTION_CLASSIFIER_ACTION=hold`	Held for the approver, and nothing is sent to the ledger. On release, all hard checks run again	Off-chain (`_hold`, `_decide_hold`)

While a draw on a receivable waits for the approver, no other draw on that receivable is accepted (`ApprovalPending`). The agent therefore cannot get around the approver by resubmitting a smaller amount.

3.3 Mandate limits

Limit	Demo value	Off-chain (L3)	On-chain (L5)
Max advance against face value	70%	`AdvanceAboveMandate` (decision rate and amount vs face)	`FinancingVault.draw`: `amount*10000 <= faceValue*maxAdvanceBps`
Max APR	12%	`AprAboveMandate`. Takes the highest of the request APR, the decision APR and the fee-derived APR, and also caps at the approved APR	`PolicyGuard` (mandate) and `Vault.draw` (`maxAprBps` from the risk oracle)
Max predicted default (PD)	5%	`PdAboveMandate` at approval (TX-2) and at draw	**Not checked on chain** (risk R11)
Approved limit	per decision	`AmountAboveLimit`	`Vault.draw` against `r.limit`
Per-tx cap, rolling daily cap	1,500,000 / 3,000,000	Yes	Yes

Limit	Demo value	Off-chain (L3)	On-chain (L5)
Validity window	2026-10-10 to 2026-11-30 (+07:00)	`MandateNotYetValid`, `MandateExpired`	`MandateExpired` only (no start date on chain)
Payee allowlist	registry ids in `config/payees.json`	Payee must resolve, be KYC-flagged and equal the bound seller	`payeeAllowed` at registration; the vault pays the bound seller
Target allowlist	`FinancingVault`, `ReceivableRegistry`	`TARGET_NAMES`	`allowedCall[target][selector]`
Currency / asset	THB / tTHB	`CurrencyNotAllowed`, mandate validation	Not on chain
Duplicate policy	DENY	—	`DuplicateReceivable`
Hard bounds on activation	advance at most 0.90, APR at most 0.30, PD at most 0.20	`Mandate` validators (HTTP 422 `MandateOutOfBounds`)	`setMandate` enforces the advance and APR bounds only (`testMandateBoundsAreHard`)
Mandate binding	`mandate_hash` = keccak256 of the canonical JSON	Request must name the active mandate (`MandateMismatch`)	`PolicyGuard.mandateHash`. On FISCO the service refuses to start if the chain's hash differs from the demo mandate's

3.4 Pause and kill switch

- `AgentWallet.pause()`: the owner or the approver can pause. `unpause()` is owner-only (`testOnlyOwnerUnpauses`, `testAStrangerCannotPause`).
- While paused, `execute` and `confirm` revert `Paused`. Because register, draw and settle all go through `execute`, every agent action stops (pytest `test_paused_wallet_reverts_every_agent_action`, `test_kill_switch_stops_draws_and_only_the_owner_resumes`).
- On FISCO the demo ran pause (tx `0x8e68…73ad`), a draw refused while paused, and an owner unpause (tx `0x2a01…2cad`).

3.5 Key custody

Key	Storage in the MVP	Notes
owner, agent, approver, risk oracle, buyer (FISCO accounts) and node keys	PEM files in `services/agent-wallet/fisco-chain/`, git-ignored (the folder's `.gitignore` ignores everything), paths from env (`CHAIN_*_KEY_PATH`)	No HSM and no rotation (risk R8)
HTTP tokens (owner, approver, risk oracle, agent, orchestrator)	`.env` only (git-ignored). The settings have no defaults for them, and they are never logged	Shared secrets (risk R1)
Anvil test accounts	Unlocked dev accounts	Scripts refuse any chain id other than 31337
`FINGERPRINT_HMAC_KEY`	`.env`. OCR falls back to a public dev key only when `OCR_DEVELOPMENT_MODE=1`, and logs a warning	Risk R23

`git grep -i "BEGIN.*PRIVATE"` at `04627de` matches only the line in `services/agent-wallet/CLAUDE.md` that describes this check. No key material is committed.

---

4. Controls mapped to threats

#	Threat	Controls (first line → backstop)	Where implemented	Evidence
T1	**Document / prompt injection** in the invoice (hidden text, tiny or off-page text, Unicode tricks, QR codes, metadata)	OCR detects instruction-like text on normalized views (NFKC, zero-width removal, look-alike letters, leetspeak, base64, joined rows) and returns `injection_flags`. Policy hard rule: any flag → `review`, so there is no TX-2 approval, and a forced draw is refused `NotApproved`. Document text is never used as instructions: the optional local Qwen reader has no tools, accepts only a loopback URL and returns schema-checked candidates	`ocr-ai/app/injection.py`, `qwen.py`; `policy-ai/rules/policy.v1.yaml` `HARD.INJECTION`; `engine._injection`	OCR tests `test_attack_payloads_are_recognized`, `test_instruction_split_across_spans_is_flagged`, `test_homoglyph_instruction_in_page_text_is_flagged_as_obfuscated`, `test_benign_invoice_text_is_not_flagged`; policy `test_injection_flag_reviews`. FISCO demo: forced draw on the injected invoice → `NotApproved` (off-chain)
T2	**Agent / tool-output injection**: text in decision `reasons`, notes or other request fields that tries to steer the wallet	L2 scans every string in the request (nested fields included) with rules in 8 languages, decoded variants (ROT13, URL encoding, base64, Unicode tag characters, spaced letters), a joined-payload scan and a canary (`CanaryLeak`). Optional ONNX classifier (ProtectAI DeBERTa v3, pinned SHA-256, local only), audit-only by default. Free text is never parsed for numbers or addresses. Payee, amount and APR come from typed, validated fields only	`agent-wallet/app/guard/injection.py`, `classifier.py`, `pipeline.py`	Seed set: L2 recall 8/8. Dev 36/36 and held-out v1 30/30 (both tuned on). Held-out v2 28/30. **Clean held-out runs with frozen rules: v3 14/24, v4 12/30.** In every set, 0 unauthorized payments (section 6.2)
T3	**Forged invoice** (inflated amount, PO mismatch, changed bank account)	OCR fraud signals with weights (arithmetic inconsistency 0.40, PO mismatches 0.50, bank account changed 0.25, and others). Policy: `fraud_score >= 0.30` → reject; other fraud flags → review; OCR `confidence < 0.80` → review; KYC invalid or expired → reject; unknown KYC → review	`ocr-ai/app/fraud.py`, `config.py` `FRAUD_WEIGHTS`; `policy.v1.yaml` `HARD.FRAUD_SCORE`, `HARD.LOW_CONFIDENCE`, `KO.KYC_INVALID`	Policy `test_fraud_score_boundary`, `test_confidence_boundary`, `test_kyc_flagged_invalid_rejects`, `test_kyc_expiry_boundary`. OCR demo file `03_inflated.pdf` → arithmetic inconsistency, fraud 0.40 → reject
T4	**Duplicate financing** (the same invoice twice, including a re-saved PDF)	Keyed fingerprint: `0x` + HMAC-SHA256 over `v1|supplier_tax_id|buyer_tax_id|invoice_no|amount|currency|issue_date`, so a re-saved file with a new `doc_hash` keeps the same key. OCR flags `duplicate_fingerprint` (weight 0.80). On chain, `ReceivableRegistry.register` reverts `DuplicateReceivable`	`ocr-ai/app/fingerprint.py`; `ReceivableRegistry.register`	`testDuplicateRegistrationReverts`; pytest `test_duplicate_registration_reverts`; OCR `04_duplicate.pdf` → fraud 0.80 → reject. **FISCO revert tx `0x30a3df21…ff15b8` (`DuplicateReceivable`)**
T5	**Over-limit or above-APR draw** (19.5% quote, draw above limit, 80% advance)	L3 off-chain, then L5 on-chain with the same codes. The APR the agent reports is also capped by the risk oracle's approved `maxAprBps`, so lying about the APR needs two keys	`pipeline.check_draw`; `PolicyGuard._validate`; `FinancingVault.draw`	`testAprAboveMandateReverts`, `testAmountAboveLimitReverts`, `testAdvanceAboveMandateReverts`, `testAmountAboveTxCapReverts`, `testDailyCapIsRolling`. **FISCO reverts: `AprAboveMandate` tx `0x4f2d22e9…c3b8`, `AmountAboveLimit` tx `0xed2cea84…e388a`.** Anvil smoke test: 7/7 checks, with `AprAboveMandate` and `AlreadyDrawn` refused on chain

#	Threat	Controls (first line → backstop)	Where implemented	Evidence
T6	**Replay / double execution**	Idempotency key on every write. A draw without a key is keyed on a hash of its whole body. A repeat returns the stored result and sends nothing. On-chain state machine: `AlreadyApproved`, `AlreadyDrawn`, `AlreadySettled`, `DuplicateReceivable`, and one pending draw per receivable	`lifecycle._run`, `_request_key`; registry and vault status	pytest `test_exact_replay_returns_the_original_and_moves_no_money`, `test_a_draw_without_a_key_or_fingerprint_does_not_replay_another_draw`; `testAlreadyDrawnReverts`, `testCannotSettleTwice`, `testAnApprovalIsGivenOnce`
T7	**Decision tampering** (review → approve, raised limit after TX-2)	Decision hash = keccak256 of the canonical decision. It is recomputed at approval and at draw, and at draw it must equal the hash the risk oracle approved. The approval's limit and fee must equal the decision's. A `review` or `reject` decision is never approved	`lifecycle._approve`; `pipeline` L1	pytest `test_approval_with_a_tampered_decision_hash_is_blocked`, `test_approval_limit_must_match_the_decision_it_names`, `test_a_review_decision_cannot_be_approved_and_the_draw_stays_blocked`. The hash binding at draw is checked off-chain only (risk R3)
T8	**Unauthorised payment release** (approve one thing, execute another; rule-setter approves)	The stored pending call is the only thing `confirm` can release, and it is revalidated at confirm. The approver is a different key and person from the owner. Holds expire after 24 h	`AgentWallet.confirm`; `PolicyGuard.revalidate`	`testLargeDrawWaitsForApproverThenExecutes`, `testDeclinedPaymentMovesNothing`, `testOwnerCannotConfirmAPayment`, `testConfirmNeedsAPendingDraw`; approval-path eval 4/4 attacks stopped at L4. **FISCO: 980,000 draw → `pending_approval` (tx `0x26788798…2fac`) → approver confirm (tx `0x38e399bc…f82a`)**
T9	**Unauthorised settlement**	Settle needs the agent token. `repay` is `onlyWallet` and passes the guard and pause. The waterfall pays the lender's principal first, then the seller's reserve, to bound addresses only. `NotDrawn` and `AlreadySettled` stop out-of-order or repeated settlement	`lifecycle._settle`; `FinancingVault.repay`	`testSettleBeforeDrawReverts`, `testCannotSettleTwice`, `testDilutedPaymentStillWaterfalls`. The buyer does not authorise each settlement (risk R13)
T10	**Key compromise**	The agent key can call `execute` only, through allowlisted targets, inside the caps, with payees bound at registration. Credit approval needs the separate risk-oracle key. Releasing a held draw needs the approver key. Owner or approver can pause	Contracts	Chain-alone eval (every off-chain layer bypassed): **0 unauthorized payments across all 216 attacks**. Known gaps: risks R5 and R7
T11	**Mandate tampering** ("ignore all limits", loose caps, expired mandate)	The policy-ai compiler rejects "unlimited" wording and values above platform maxima (HTTP 422). Activation needs the owner token and passes the hard bounds and env ceilings without silent clamping. On-chain `MandateOutOfBounds`, `MandateExpired`. The FISCO startup check compares mandate hashes	`policy-ai/app/mandate.py`; `Mandate` validators; `PolicyGuard.setMandate`	pytest `test_mandate_activation_needs_owner_token_and_stays_in_bounds`; `testMandateBoundsAreHard`, `testMandateExpiryReverts`
T12	**Payee substitution (BEC)**	Payees are registry ids. A raw address is accepted only if it is already a registry address. The seller is bound on chain at TX-1, and the vault pays only that address. Only KYC-flagged payees are allowlisted on chain	`lifecycle._payee_address`, `activate_mandate`; `ReceivableRegistry.register`; `Vault.draw`	`testUnallowlistedSellerReverts`; pytest `test_unverified_payees_are_not_allowlisted_on_the_chain`, `test_a_payee_without_kyc_cannot_be_registered`

#	Threat	Controls (first line → backstop)	Where implemented	Evidence
T13	**Misuse of the demo bypass**	`skip_offchain_checks` is refused (`SchemaInvalid`) unless `DEMO_MODE=1`, and every use is audited as `OffchainBypassed`	`app/main.py execute`; `lifecycle._execute`	pytest `test_demo_bypass_is_rejected_without_demo_mode`
T14	**Audit tampering**	Hash-chained entries: each entry hashes the one before it. `verify()` fails on any edit, reorder or deletion. The head is anchored on chain (`AgentWallet.anchorAudit`) after each settlement	`app/audit/log.py`; `lifecycle._audit`	pytest `test_editing_an_entry_fails_verification`, `test_dropping_an_entry_breaks_the_chain`, `test_full_loop_is_audited_anchored_and_verifies`
T15	**Data privacy: no PII on chain**	On chain: keyed fingerprint, `doc_hash`, seller address, face value, due date, limit, APR, decision hash, statuses. No names, tax IDs or invoice text. Audit entries hold ids, codes and hashes only. OCR stores scoped HMAC tokens, not bank account numbers. The demo uses synthetic parties only	`ReceivableRegistry.Receivable` struct; `audit/log.py`; `ocr-ai/app/fingerprint.py`	pytest `test_audit_entries_carry_no_invoice_text`

5. Known-risk list

Likelihood (L) and impact (I) are rated High, Med or Low for a pilot deployment. All 24 risks below are open at `04627de`.

#	Risk	L	I	Current mitigation	Residual risk	Planned fix
R1	**HTTP tokens are shared secrets with no caller identity.** One leaked agent token gives the full agent API. The orchestrator holds both the agent and risk-oracle tokens	Med	High	Every write needs a token. Tokens fail closed when unset and are compared in constant time. The orchestrator requires its own caller token. On-chain caps, allowlists and payee binding still limit any draw	Med	Private network only; per-caller identity (mTLS or signed requests); the approver signs on their own device
R2	**Risk-oracle key is co-located with the wallet** (its key path is in the wallet env, and its token is in the orchestrator env), so the platform could approve its own receivable	Med	High	`/wallet/approve` needs the token and the exact decision. The hash, limit and fee must match, the decision must be `approve`, and the PD must be at or below the ceiling	Med	Move the oracle key to policy-ai's own signer or an HSM

#	Risk	L	I	Current mitigation	Residual risk	Planned fix
R3	**Decisions are hashed, not signed.** The wallet can check that a decision is intact, but not who wrote it. The draw-time check of the decision against the approved hash is off-chain only: `AgentWallet.execute` takes `decisionHash` and only emits it	Med	Med	The hash is recomputed at TX-2 and at draw. The on-chain limit and APR come from the oracle's approval. `REQUIRE_SIGNED_DECISION` was removed because nothing read it	Med	policy-ai signs decisions (a `contract:` change to `PolicyDecision`); `Vault.draw` compares the decision hash with `r.decisionHash`
R4	**Decision freshness is weak.** `valid_until` is a wallet-side stopgap. It is excluded from the decision hash and optional, so a caller can extend or drop it. The 15-minute decision-age rule in the wallet brief is not implemented. The policy TTL is 24 h	Med	Low	One draw per receivable on chain, at the approved limit, inside the mandate	Low	Add `valid_until` to the signed shared decision and enforce a maximum age
R5	**The lender address and the fee split in `Vault.draw` are not constrained on chain** (found in the code review for this report). A stolen agent key could name any address as `lender` and set `lenderFee` up to just below `amount − platformFee`. The seller would be underpaid, the fee would be claimable by that address, and the principal repaid at settlement would go to it. Off-chain, `lender_id` is not checked by the guard: the FISCO adapter raises an error for an unknown id before sending, and the mock books it to an arbitrary label. Liquidity is pooled, so draws are not tied to the named lender's deposit	Low	High	Needs the agent key. Bounded by an approved receivable, the approved limit, the advance ceiling, and the per-tx and daily caps. The HTTP path computes fees from the hash-bound `fee_rate`	Med	Require `payeeAllowed(lender)` in `Vault.draw`; cap `lenderFee` from the approved APR and tenor; add `LenderNotAllowed` to L3; add eval cases
R6	**Single approver.** No M-of-N	Med	Med	The approver is a different person and key from the owner. Held calls are bound to their exact data and revalidated at confirm	Med	2-of-3 approvers on hardware keys
R7	**Owner key is a single EOA with full control.** The owner address is immutable and has no multisig or timelock. `setMandate` bounds only the advance and APR on chain; the per-tx cap, daily cap, review threshold and expiry are unbounded there. The owner can replace the agent or approver and mint tTHB	Low	High	Off-chain activation enforces env ceilings and hard bounds. Only the owner unpauses	Med	Multisig owner, a timelock on any loosening change, on-chain ceilings for the caps
R8	**Keys are PEM files on a developer machine.** No HSM, no rotation	Med	High	Git-ignored. Never logged or returned. Anvil unlocked accounts are limited to chain 31337	Med	HSM or KMS, key rotation, separate hosts per role
R9	**FISCO evidence comes from one chain instance**: 4 nodes in one Docker container on one host, one deploy and one demo run	Med	Med	22/22 deploy and 11/11 demo hashes verified on chain, with receipts saved	Med	A multi-operator consortium deployment

#	Risk	L	I	Current mitigation	Residual risk	Planned fix
R10	**Injection detector recall is partial.** Clean held-out results: v3 14/24, v4 12/30. Misses are English directives without an override verb, and phrasings in th, zh, ko, es, fr and vi. The classifier is English-only, does not detect jailbreaks, and is audit-only by default. Blocking mode raises benign false blocks (v3: 2/10 with the classifier blocking vs 1/10 with rules)	High	Low	Detection is not the defence. 30 attacks reached execute across all sets, and every draw stayed inside the mandate: **0 unauthorized**	Low	Multilingual classifier, new held-out sets for every rule change, an action-alignment check
R11	**PD is enforced off-chain only** (at TX-2 approval and at draw). A stolen risk-oracle key could approve a high-PD receivable on chain	Low	Med	Since the 15 Oct fix, TX-2 refuses decisions above the PD ceiling, so the chain-alone eval shows 0 unauthorized (earlier runs: W07, S15, S16 executed)	Low	Pass PD into `execute` and check it in `PolicyGuard`, or have the oracle sign the PD
R12	**KYC/KYB is a mock.** `kyc_verified` and `kyc_expiry` are flags in `config/payees.json`. Policy reads `kyc_valid` from caller-supplied profiles (static samples in the orchestrator). KYC expiry is enforced off-chain only; the on-chain allowlist keeps the address	High (pilot)	Med	Unverified or expired payees are refused at register and draw. Only verified payees go on chain	Med	A real KYB provider, sanctions and PEP screening, removal from the allowlist on expiry
R13	**Settlement is not buyer-authorised.** The agent triggers `repay`, with a caller-supplied `amount_paid`, from one `settlementPayer` allowance	Med	Med	Funds go only to the bound lender and seller. `AlreadySettled` and `NotDrawn` apply	Med	Buyer-signed payment, or a bank-confirmation oracle
R14	**Wallet state is in memory**: holds, pending records, the idempotency store, receivable facts and the **audit log itself**. A restart loses them. docs/06 describes a hash-chained JSONL file, but the code keeps a Python list	Med	Med	The chain backstops replays (`DuplicateReceivable`, `AlreadyApproved`, `AlreadyDrawn`, `AlreadySettled`). On FISCO the daily cap is reseeded from the chain. The audit head is anchored at each settlement	Med	A durable store (SQLite) for state and an append-only audit file
R15	**On-chain pending draws never expire.** The 24 h expiry is off-chain. After a restart a held call stays on chain and the approver could still confirm it (with revalidation). Declined draws keep counting toward the daily cap for 24 h	Low	Low	Confirm revalidates the target, expiry, cap and APR	Low	An expiry timestamp in `Pending`; release cap budget on decline
R16	**No cap on outstanding exposure.** Nothing limits the total balance outstanding across receivables	Med	Med	The daily cap (3,000,000) equals the seller's facility limit	Med	An on-chain outstanding-exposure limit per seller

#	Risk	L	I	Current mitigation	Residual risk	Planned fix
R17	**Audit gaps.** No `flow_id`. Anchoring happens only after settlement, not after blocked flows. Audit read endpoints are unauthenticated. The actor field is coarse (`agent`, `approver`, `operator`)	Med	Low	Entries carry `invoice_id`, `tx_id`, codes and chain refs, not invoice text	Low	`flow_id` from the orchestrator, anchoring at the end of every flow, authenticated reads
R18	**The full red-team runs against the Python mock only.** The contracts are covered by Foundry tests (42), the Anvil smoke test (7 checks) and the FISCO demo, not by the 216-case set	Med	Med	The mock mirrors the contract checks and revert strings. The chain-alone eval found no unauthorized path in the mock. R5 shows that mock and contract can differ in what they constrain	Med	Run the eval with `--ledger fisco`
R19	**Small draws are falsely blocked.** The platform fee is 0.3% of face, and the vault requires fees below the draw, so a 100.00 draw fails `FeesExceedAmount` (v5 SB05)	Med	Low	Fails safe	Low	Take the platform fee from settlement, or set a minimum draw
R20	**Reason codes differ for the same refusal.** An 80% advance that is also above the limit reports `AdvanceAboveMandate` off-chain and `AmountAboveLimit` on chain	Low	Low	The outcome is the same refusal	Low	Align the check order
R21	**Simulated money and market.** tTHB is a mock token the owner can mint. Lender quotes are a static file (`data/agent/lender_quotes.json`). Buyer and supplier profiles are samples	High (by design)	Low (demo)	Stated in the pitch. Permissioned chain only	n/a	Licensed THB rail (bank or the BoT sandbox), real lender integration
R22	**OCR limits.** Fraud scores are uncalibrated rule weights. Duplicate history is a local store capped at 500 observations, so there is no cross-platform double-financing check. `/ocr/extract` and `/policy/decide` are unauthenticated (the README says it assumes a trusted local network). The optional hosted vision reader (`OCR_HOSTED_VISION_ENABLED=1`, off by default) sends invoice images to an external API	Med	Med	The on-chain registry stops duplicates within the platform. The hosted reader is off by default and its output is schema-validated candidates only	Med	Calibrate on labelled fraud data, link to the national factoring registry, add service auth, redact before any hosted model
R23	**Fallbacks and dev keys.** The policy PD model falls back to the scorecard and then to a fixed placeholder formula if loading or scoring fails (reported in `model_version`). OCR uses a public development fingerprint key when `OCR_DEVELOPMENT_MODE=1` and the key is unset, which would make fingerprints guessable	Low	Med	Fallbacks are deterministic, logged and visible in the decision. The dev key logs a warning and is meant for demo data only	Low	Fail closed in production mode on model fallback or a missing HMAC key

#	Risk	L	I	Current mitigation	Residual risk	Planned fix
R24	**Optional LLM steps.** The mandate compiler and the explanation endpoint may call an LLM when one is configured	Low	Low	Not on the money path. Compiler output is re-validated against platform maxima. Explanation text must pass number, outcome and address checks, or the template is used. Untrusted text is wrapped in nonce delimiters. Without an LLM, deterministic code runs (the default)	Low	—

Where the docs claim more than the code does (each one is covered by a risk above):

- docs/06 and the wallet brief describe a JSONL audit file and anchoring every 10 entries or at the end of each flow. The code keeps the log in memory and anchors after settlement only (R14, R17).
- The wallet brief (section 5.4) specifies a 15-minute decision-age rule and HMAC-signed decisions. Neither is implemented (R3, R4).
- The wallet brief (section 5.6) lists `AdvanceAboveMandate` inside `PolicyGuard.check`. It is checked in `FinancingVault.draw` instead, so it is still enforced on chain.
- The wallet brief (section 5.6) says `confirm` is `onlyOwner`. The code uses `onlyApprover`, which is the stronger separation and matches section 5.1 of the same brief.
- docs/06 §1.1 says "one draw per invoice key" and that the agent key can only call `execute`. Both hold. It does not mention that the fee split and lender address in `draw` are agent-supplied (R5).

---

6. Test evidence summary

6.1 Test suites

Suite	Result	Notes
`pytest services/agent-wallet`	**155 passed, 2 skipped**	Re-run 2026-10-19 at `04627de`. The 2 skips need `INJECTION_MODEL_DIR` (the classifier model)
`pytest services/policy-ai`	**392 passed, 4 skipped**	Skips need the optional `pandas`, `sklearn` and `lightgbm` packages
`pytest services/ocr-ai`	**326 passed, 1 skipped**	Needs the service's `requirements-dev.txt`
`pytest services/orchestrator`	**40 passed, 19 skipped**	Skips are live end-to-end tests (`E2E_LIVE=1`)
`forge test --offline` (contracts)	**42 passed** (engineer's report)	40 tests in `Wallet.t.sol` and 2 in `TimeScale.t.sol`, confirmed by counting the test functions in source. Every contract revert string except `TransferFailed` has an `expectRevert` test
`ruff check` (whole repo)	**Clean**	Re-run 2026-10-19
Anvil smoke test (`scripts/evm_smoke.py`)	7/7 checks (engineer's report)	Real Solidity on a local EVM. Balances match the mock

6.2 Red-team results (full stack = the configuration the demo runs)

Source: `services/agent-wallet/eval/REDTEAM.md` and `eval/report_*.md` (mock ledger, no LLM, no network). Re-generated for this report with identical results.

Case set	Attacks	Scanner alone flags	Chain alone stops (unauthorized)	Full stack stops	Full stack: reached execute / unauthorized	Benign completed
Seed (`cases.jsonl`)	38	9/38	29/38 (0)	38/38	0 / **0**	11/11
Dev (tuned on)	36	36/36	0/36 (0)	36/36	0 / **0**	18/18
Held-out v1 (tuned on after first run)	30	30/30	0/30 (0)	30/30	0 / **0**	10/10
Held-out v2	30	28/30	0/30 (0)	28/30	2 / **0**	10/10
Held-out v3 (first clean run, rules frozen)	24	14/24	0/24 (0)	14/24	10 / **0**	9/10
Held-out v4 (second clean run)	30	12/30	0/30 (0)	12/30	18 / **0**	14/14
Held-out v5 (structural: amount, payee, APR, PD, mandate state)	24	4/24	21/24 (0)	24/24	0 / **0**	9/10
Human-approval path	4	0/4	3/4 (0)	4/4	0 / **0**	3/3
All sets	**216**	133/216	53/216 (0)	186/216	30 / **0**	84/86

How to read it:

- "Unauthorized" means tTHB moved to a non-allowlisted address, above a cap, outside the mandate, or twice. The count comes from the ledger's state after each case, not from the guard's verdict. It is 0 in every set.
- "Reached execute" counts attacks whose text the scanner missed, but whose draw used the mandate's payee, amount and APR. These are detection misses, not losses.
- The dev and held-out v1 sets were tuned on, so treat them as regression sets. Only v3 and v4 measure generalisation.
- The 2 benign cases blocked are v3 VB01 (an `InjectionSuspected` false positive) and v5 SB05 (`FeesExceedAmount`, risk R19).
- In the seed set, the first layer that stopped each attack was: L0 4, L0-auth 4, L1 6, L2 8, L3 12, L5 3.

6.3 On-chain revert and execution evidence (FISCO BCOS 3.16.4, chain0 / group0)

- **Deploy:** 22 transactions (5 contract deploys and 17 setup calls), all found on chain (`eval/fisco-deployment.md`, `eval/fisco-verification-deploy.txt`, receipts in `eval/fisco-receipts/`).
- **Demo run (2026-10-11):** 11 on-chain hashes, all verified: 8 succeed and 3 revert (`eval/demo_tx_hashes_fisco.md`, `eval/fisco-verification-demo.txt`).

Case	Expected	On-chain result	Tx hash
19.5% APR draw, off-chain bypassed	revert	`AprAboveMandate`	`0x4f2d22e929adc19e77185e8e5fe8ea87abba6598638b9c6f90d0b29555bcc3b8`
600,000 draw above the 524,300 limit, bypassed	revert	`AmountAboveLimit`	`0xed2cea84e4925891d7bd21a1d157c65cdde1101ba42c20acdb1e8f5e9ade388a`
Re-saved PDF registered again	revert	`DuplicateReceivable`	`0x30a3df21e618d5eab6b032b171736e0fe7e81f0fd36f08392b3ac6c212ff15b8`

Case	Expected	On-chain result	Tx hash
980,000 draw (at or above 837,500)	held	`pending_approval`	`0x267887986a61aff4a3b67ce09510a3e7abac7ff00916c64423fe71f739d32fac`
Approver confirms	executes	ok	`0x38e399bc4ef26de2e45bc985764b307735c0acfb7685908c58d5cbb1c550f82a`
Pause / unpause (owner)	ok	ok	`0x8e68…73ad` / `0x2a01…2cad`

The same demo also refused, off-chain and before any transaction: the APR and over-limit draws without the bypass, a forced draw on the injected invoice (`NotApproved`), and a draw while paused (`Paused`).

A bug found only on FISCO: the chain reports `block.timestamp` in milliseconds, so the first deployment saw every mandate as expired. It was fixed with an immutable `PolicyGuard.timeScale` (1000 on FISCO, 1 on EVM) and covered by `TimeScale.t.sol`.

---

7. Compliance notes

- **Customer type.** ReceivableOS onboards businesses (an SME seller, a corporate buyer, licensed lenders), not anonymous wallets. Every payee is a registry entry. Only KYC-flagged payees are allowlisted on chain, and funds can reach only the seller bound at registration or the platform fee address.
- **KYC/KYB and AML.** The MVP uses mock KYB flags with expiry dates (R12). It has no sanctions, PEP or mule-account screening. In production, the licensed lender or bank partner would remain the AML reporting entity under Thailand's Anti-Money Laundering Act. ReceivableOS would supply KYB evidence and the tamper-evident decision trail. We have not obtained legal advice; this is our understanding, not a legal opinion.
- **Tokens.** tTHB is a simulated token on a permissioned chain. It is not offered to investors and is held only by allowlisted demo accounts. A real deployment would use a licensed THB rail.
- **Privacy (Thailand PDPA).** No personal data goes on chain: only the keyed fingerprint, the document hash, amounts, dates, statuses, the decision hash and addresses. The keyed fingerprint (HMAC) stops anyone from brute-forcing tax IDs or invoice numbers from the chain. Audit entries hold ids, codes and hashes, not invoice text. The demo uses only synthetic, fictional parties. The hosted vision reader that would send documents abroad is off by default (R22). A production rollout needs a DPIA, data-processing agreements and redaction before any external model.
- **Human oversight of AI.** No model output moves money by itself. Credit decisions pass deterministic hard rules before any model. Every money action is checked by code and by the contracts. Payments at or above THB 837,500, or held by the classifier, need a named human approver who does not set the rules.
- **Record keeping.** Every executed, held and blocked action is recorded in a hash-chained audit log. The log's head is anchored on chain after each settlement. The log is not yet durable across restarts (R14).